

Trabajo Práctico Airflow: Dags

Santiago Arena

Ingeniería de Datos

Universidad Católica Argentina
Facultad de Ingeniería y Ciencias
Agrarias

Noviembre 2025

Índice

- **1. Introducción**
- **2. Exploración y parámetros**
 - 2.1 Presentación de los datasets
 - 2.2 Transformación del dataset de temperatura
 - 2.3 Transformación del dataset de calidad del aire
- **3. DAG**
 - 3.1 Arquitectura modular del pipeline
 - 3.2 Descripción de las tareas (`air_quality_alert_email`, `combina_data`, `download_data`, etc.)
 - 3.3 Ejecución diaria y parámetros principales
 - 3.4 Flujo completo del proceso (Figura 1)
 - 3.5 Artefactos generados
- **4. Resultados**
 - 4.1 Ejecución exitosa del DAG (Figura 2)
 - 4.2 Notificaciones por correo electrónico (Figura 3)
 - 4.3 Alerta por contaminantes críticos (Figura 4)
 - 4.4 Ejemplos de excesos respecto del P95
 - 4.5 Notificación de finalización del pipeline (Figura 5)
- **5. Conclusiones**
- **6. Bibliografía**

1 Introducción

2 Exploración y parámetros

2.1 presentación de los datasets

Los datos se descargan automáticamente en la carpeta **data** al momento de ejecutar el pipeline. En esta etapa se generan dos archivos crudos: **temperature_raw.csv** y **air_quality_raw.csv**, los cuales sirven como punto de partida para las transformaciones posteriores.

El dataset **Temperature** contiene 354 filas y 5 columnas, cubriendo el periodo 1991–2020. Se presenta en un nivel de agregación mensual y sus columnas son: **año**, **mes**, **máxima**, **mínima** y **media**. El archivo llega en formato CSV utilizando “;” como separador, lo cual es habitual en exportaciones oficiales del Gobierno de la Ciudad.

Por su parte, el dataset **Air Quality** incluye 139.068 filas y 14 columnas, abarcando mediciones horarias desde 2009 hasta 2025. Su granularidad es efectivamente horaria, con aproximadamente 10 registros por hora por contaminante debido a la presencia de múltiples sensores. Se reciben mediciones procedentes de las estaciones de monitoreo ubicadas en Centenario, Córdoba, La Boca y Palermo. Este archivo también se presenta en formato CSV, manteniendo la estructura original provista por la Agencia de Protección Ambiental.

2.2 Temperatura

En el presente inciso se describen de manera sintética las transformaciones aplicadas al dataset de temperatura. En primer lugar, se realiza la lectura del archivo **csv** utilizando el separador “;”. La librería **pandas** maneja este formato sin inconvenientes. Las columnas originales vienen codificadas en UTF con tildes, lo cual puede generar inconsistencias entre sistemas; para evitarlo, se asignan los nombres de columna siguiendo una lógica posicional, dado que la estructura del archivo es estable en el tiempo.

Se aplica posteriormente una validación de rangos basada en valores históricos publicados por el Servicio Meteorológico Nacional (referenciados en Wikipedia). Toda temperatura superior a 50°C o inferior a -10°C se considera fuera de rango. Si bien se trata de un criterio rígido, resulta confiable para series mensuales. Estas detecciones se registran en un arreglo (que en todas las ejecuciones resultó vacío).

Respecto del tratamiento de datos faltantes, se eliminan únicamente aquellas filas que son completamente nulas. Luego, mediante mapeos simples, se generan una columna de mes numérico y otra de mes en texto normalizado. También se construye la columna **fecha** en formato estándar y el entero correspondiente al Día del Año (DOY).

En caso de que alguna etapa falle, el sistema retorna un **DataFrame** compuesto por valores nulos, asegurando así la continuidad del pipeline. Además,

se registra un timestamp con el momento exacto en que finaliza la transformación. Finalmente, el archivo transformado se guarda en la misma carpeta que el archivo original.

Se considera como clave primaria natural del dataset la combinación de las columnas `anio` y `mes`.

2.3 Calidad del Aire.

El dataset se extrae desde la carpeta `data` y se presenta originalmente con granularidad horaria, aunque con una distribución irregular (entre 3 y 4 registros por día). Además, el archivo CSV llega desordenado, por lo que el ordenamiento temporal constituye un paso inicial necesario.

En primer lugar, se eliminan las columnas con valores nulos sistemáticos (particularmente sensores en la estación Palermo). Asimismo, los valores "`s/d`" presentes en el archivo —que representan datos faltantes en formato texto— se reemplazan por `None`, garantizando compatibilidad con los tipos nativos de Python.

A continuación, la columna `fecha` se transforma a un formato estándar comprensible por Python. Esto permite extraer correctamente el año, mes y día. El mayor desafío de este paso radica en parsear el string original, pero `pandas` provee mecanismos robustos para interpretar fechas con formatos heterogéneos. Una vez resuelta la conversión, el dataset se ordena cronológicamente.

Para el proceso de agregación mensual, se realiza un *groupby* utilizando la clave compuesta `[anio, mes]`. Sobre cada grupo se calculan promedios mediante el operador `.mean()` de `pandas`, obteniendo así un resumen mensual de los contaminantes.

Durante la ejecución, se imprimen longitudes de arrays y otros indicadores (`len`, conteos, estados intermedios), lo que facilita la trazabilidad en los logs y permite verificar que los volúmenes de datos procesados sean consistentes.

El dataset mensual resultante se guarda en el mismo directorio y será utilizado posteriormente en la etapa de combinación con la serie de temperatura.

Finalmente, se implementa un sistema de alerta temprana para identificar picos de contaminación. El criterio consiste en detectar si algún contaminante supera su valor correspondiente al percentil 95 de la serie histórica. En caso de que se registre un exceso, se calcula y almacena la diferencia relativa respecto de dicho umbral, lo que permite cuantificar la severidad del evento. Este umbral puede flexibilizarse en el futuro si se requiere mayor permisividad.

3 DAG

El pipeline funciona a través de un *DAG* principal que importa y ejecuta tareas modulares. Cada tarea corresponde a un archivo `.py` independiente ubicado en la carpeta **Tareas**, lo que aporta una estructura clara y facilita el mantenimiento del proyecto. En total, el sistema cuenta con siete módulos:

- `air_quality_alert_email`
- `combina_data`
- `download_data`
- `email_logging`
- `email_operator`
- `transformation_1`
- `transformation_2`

El *DAG* coordina tanto tareas paralelas como secuenciales, garantizando la integridad del flujo de datos, la generación oportuna de análisis y la correcta distribución de los productos finales. Además, incorpora notificaciones automáticas por correo y políticas básicas de resiliencia operativa ante fallos, tal como se muestra en la Figura 1.

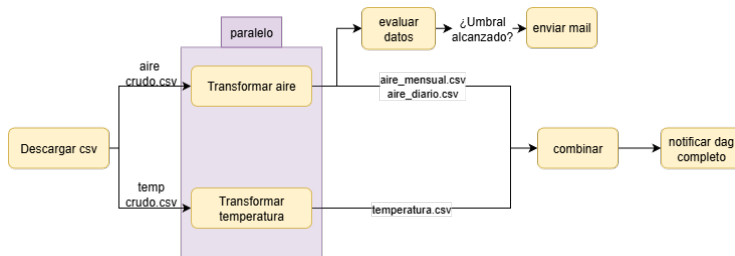


Figure 1: Flujo planteado

El dag cuenta con ejecución diaria a las 08:00 (UTC). Algunos de sus parametros son:

- **Identificador:** `dag_data_pipeline`
- **Programación:** `0 8 * * *` (ejecución diaria a las 08:00 UTC); `catchup = False`
- **Reintentos:** 1 intento adicional con espera de 5 minutos
- **Notificaciones:**

- Correo en caso de fallo o reintento
 - Correo de alerta de calidad de aire (condicional)
 - Correo de resumen del pipeline (siempre)
- **Flujo de tareas:**
 - **Ingesta:** descarga de datos brutos de calidad de aire y temperatura
 - **Transformaciones en paralelo:** limpieza y normalización de temperatura (mensual) y calidad de aire (horaria → agregado mensual y dataset diario con DOY)
 - **Alerta operativa:** evaluación del último día contra el percentil 95; si excede, se envía email de alerta
 - **Análisis mensual de contaminación:** cálculo de umbrales P95 por contaminante/estación y generación de reportes auxiliares
 - **Integración:** combinación de agregados de aire con la serie mensual de temperatura mediante clave año-mes (LEFT JOIN)
 - **Publicación y comunicación:** generación del dataset final y envío del correo de resumen del pipeline
 - **Artefactos principales:**
 - **Dataframes intermedios:** `air_quality_daily.csv`, `air_quality_monthly.csv`
 - **Producto final:** `/opt/airflow/data/producto/buenos_aires_environmental_data.csv`
 - **Reportes auxiliares:** estadísticas de ciudad, alertas históricas (P95), análisis detallado y resumen de procesamiento

4 Resultados

La Figura 2 muestra el DAG ejecutado exitosamente desde la interfaz de Airflow, evidenciando la correcta secuencia y finalización de todas las tareas.

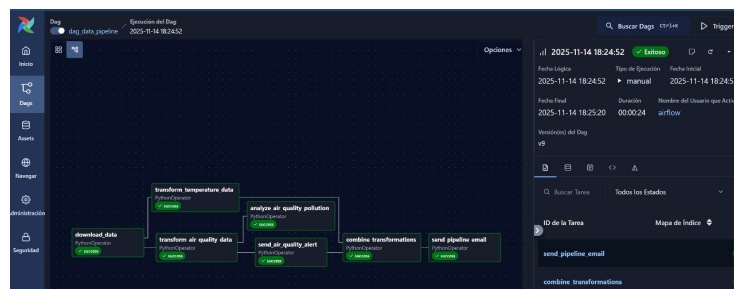


Figure 2: DAG completado exitosamente

La Figura 3 ilustra el funcionamiento adecuado del sistema de notificaciones por correo electrónico. Se observan dos mensajes recibidos: uno correspondiente a una alerta por umbrales críticos superados y otro vinculado al resumen de ejecución del DAG.

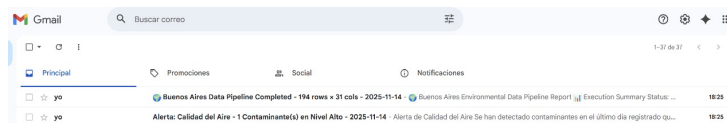


Figure 3: Recepción de correos automáticos del pipeline

En la Figura 4 se presenta el contenido del correo generado cuando se detecta un contaminante por encima del percentil 95. Para este ejemplo, la notificación fue forzada, por lo que se observa un exceso del 0% (el valor no superaba el umbral). El sistema informa también el porcentaje de exceso en relación con el P95 para cuantificar la magnitud del evento.

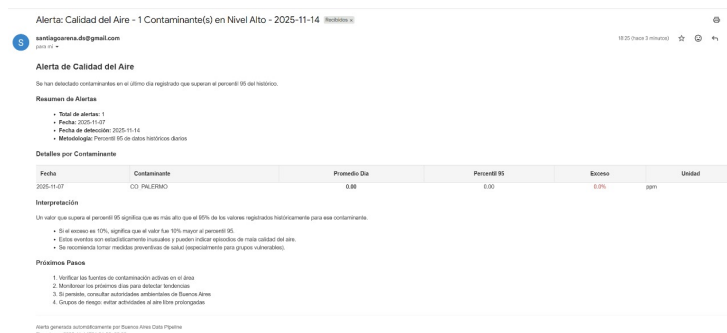


Figure 4: Correo de alerta por contaminantes críticos

A continuación se muestran algunos ejemplos reales de excesos detectados por el sistema:

- CO.CENTENARIO: 24.06 ppm vs P95 = 2.58 → Exceso 833.7%
- NO2.CENTENARIO: 33.70 $\mu\text{g}/\text{m}^3$ vs P95 = 27.12 → Exceso 24.3%
- PM10.CENTENARIO: 70.69 $\mu\text{g}/\text{m}^3$ vs P95 = 37.95 → Exceso 86.3%
- CO.CÓRDOBA: 2.22 ppm vs P95 = 1.18 → Exceso 88.1%
- NO2.CÓRDOBA: 145.00 $\mu\text{g}/\text{m}^3$ vs P95 = 50.15 → Exceso 189.1%
- PM10.CÓRDOBA: 52.47 $\mu\text{g}/\text{m}^3$ vs P95 = 36.72 → Exceso 42.9%
- CO.LA.BOCA: 1.60 ppm vs P95 = 0.88 → Exceso 82.3%

- NO2.LA.BOCA: 43.31 $\mu\text{g}/\text{m}^3$ vs P95 = 34.68 $\rightarrow$ Exceso 24.9%
- PM10.LA.BOCA: 62.37 $\mu\text{g}/\text{m}^3$ vs P95 = 42.34 $\rightarrow$ Exceso 47.3%
- CO.PALERMO: 1.36 ppm vs P95 = 0.15 $\rightarrow$ Exceso 805.9%

Finalmente, la Figura 5 muestra el correo de confirmación de finalización correcta del DAG, enviado automáticamente al cierre del pipeline. Esta corre permite resumir información clave sobre el proceso ETL. En el se podría monitorear cualquier aspecto del proceso, en este caso, dimensiones iniciales y finales coherentes al observar el shape de los datos mensuales y en el merge resultante..

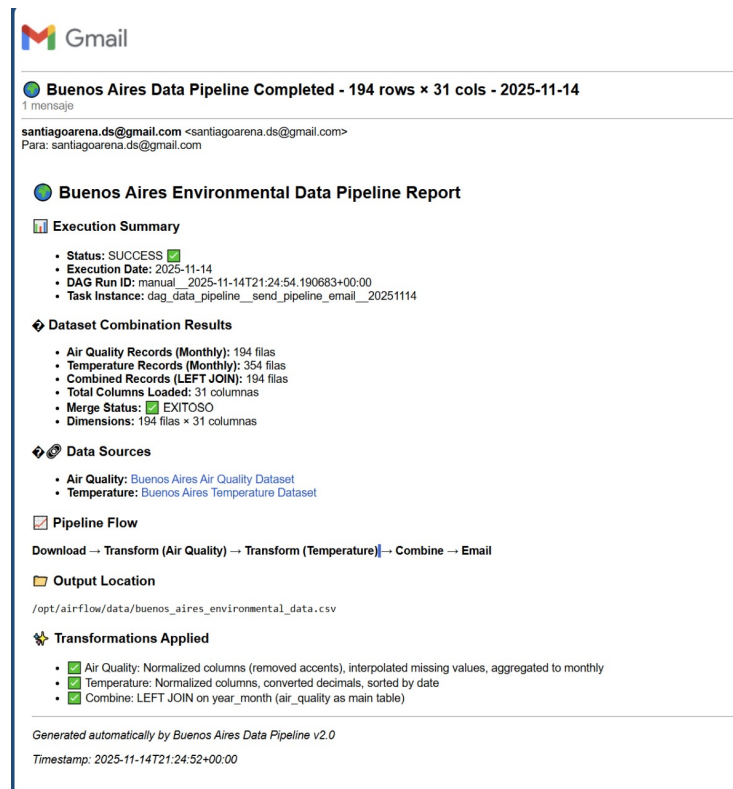


Figure 5: Notificación de ejecución completa del DAG

5 Conclusiones

El desarrollo del pipeline permitió automatizar de forma confiable la adquisición, transformación y análisis diario de datos ambientales de la Ciudad de Buenos Aires, integrando fuentes heterogéneas bajo un flujo robusto y auditable. La arquitectura basada en Airflow demostró ser adecuada para coordinar tareas paralelas, manejar fallos y ofrecer trazabilidad completa mediante logs y notificaciones por correo. Las transformaciones implementadas aseguraron la calidad de los datos mediante validaciones de rango, estandarización de fechas, interpolación y controles de consistencia. Asimismo, el cálculo de umbrales dinámicos (P95) y el sistema de alertas aportaron un componente analítico capaz de identificar episodios críticos de contaminación. En conjunto, el pipeline constituye una herramienta sólida para monitorear tendencias ambientales y servir como base para futuras ampliaciones analíticas, como modelos predictivos o dashboards interactivos.

References

- [1] Apache Airflow Documentation. *Airflow: Platform to programmatically author, schedule and monitor workflows*. Disponible en: <https://airflow.apache.org/>
- [2] Pandas Development Team. *Pandas: Python Data Analysis Library*. Disponible en: <https://pandas.pydata.org/>
- [3] Gobierno de la Ciudad de Buenos Aires. *Portal de Datos Abiertos*. Disponible en: <https://data.buenosaires.gob.ar/>
- [4] Servicio Meteorológico Nacional. *Registro histórico de temperaturas extremas*. Disponible en: <https://www.smn.gob.ar/>